

B.TECH 1ST YEAR · 1ST SEMESTER · EXAM READY

C Programming

Unit II — Control Structures

Formula Sheet & Revision Document

Last-Day Revision · Exam Guide · Memory Booster

Sequential

Selection

Iteration

if / switch

for / while / do-while

break / continue

Nested Loops

Patterns

Flashcards · Comparison Tables · Syntax Boxes · Key Terms · Exam Tips

§1 — Introduction to Control Structures

📌 Definition

A **control structure** is a block of programming that decides the **execution path** of a program based on conditions or repeated operations.

Why Control Structures?

- Programs aren't always linear — decisions and repetition are needed.
- Avoids code duplication (loops) and enables branching logic (conditions).
- Foundation of algorithm design and problem solving.

Types of Control Structures

Control Structures | — 1. Sequential — statements execute one after another (top → bottom) | — 2. Selection — execute a block ONLY IF a condition is true | — if | — if-else | — else-if ladder | — nested if | — switch | — 3. Iteration — repeat a block while/until a condition holds | — for loop | — while loop | — do-while loop

Type	Key Idea	Keywords / Syntax
Sequential	Top-to-bottom, one by one	(default flow)
Selection	Choose a path based on condition	if, else, switch
Iteration	Repeat until condition fails	for, while, do-while

§2 — Sequential Programs

Definition

In a **sequential program**, statements are executed **one after another** in the order written — no skipping, no branching, no repeating.

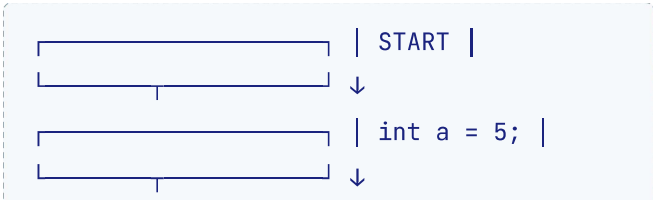
Characteristics

- Every statement is executed exactly once.
- Execution order matches source code order.
- No conditions or loops are involved.
- Simple and predictable; used for straightforward calculations.

Example Program

```
#include <stdio.h>
int main() {
    int a = 5;      // Step 1
    int b = 10;     // Step 2
    int sum = a + b; // Step 3
```

Execution Flow



```
printf("%d", sum); // Step 4  
return 0;  
}
```

	int b = 10;
	↓
	sum = a + b;
	↓
	printf(sum);
	↓
	END

§3 — Conditional Statements

(A) if Statement

```
if (condition) { // executes only if
condition TRUE statement(s); }
```

condition TRUE → execute block FALSE → skip block → continue

```
#include <stdio.h>
int main() {
    int n = 10;
    if (n > 0) {
        printf("Positive");
    }
    return 0;
}
```

(B) if-else Statement

```
if (condition) { // TRUE block
statement(s); } else { // FALSE block
statement(s); }
```

condition TRUE → if-block executes FALSE → else-block executes

```
int n = -5;
if (n ≥ 0) {
    printf("Non-negative");
} else {
    printf("Negative");
}
```

(C) Nested if

Concept

An `if` inside another `if` — used when multiple conditions must all be true simultaneously.

```
int a = 10, b = 20, c = 30;
if (a > b) {
    if (a > c) {
        printf("a is largest");
    }
} else {
    if (b > c) {
        printf("b is largest");
    } else {
        printf("c is largest");
    }
}
```

(D) else-if Ladder

Use case

When there are **multiple mutually exclusive conditions** to test in sequence.

```
if (c1) { ... } else if (c2) { ... } else
if (c3) { ... } else { // default }
```

```
int marks = 75;
if (marks ≥ 90)    printf("A");
else if (marks ≥ 75) printf("B");
else if (marks ≥ 60) printf("C");
else               printf("F");
```

(E) switch Statement

```
switch (expression) { case val1:
statement(s); break; case val2:
statement(s); break; default:
statement(s); }
```

Rules

- `break` stops fall-through to next case.
- Without `break`, all subsequent cases execute (fall-through).
- `default` runs when no case matches.

- Expression must be **integer or char**, not float or string.
- Duplicate case values are NOT allowed.

```
int day = 2;
switch(day) {
    case 1: printf("Mon"); break;
    case 2: printf("Tue"); break;
    default: printf("Other");
}
```

if-else vs switch — Comparison

Feature	if-else	switch
Condition Type	Any relational / logical expression	Only integer or character constants
Flexibility	High — ranges, complex conditions	Limited — exact equality only
Readability	Can get messy with many conditions	Clean and readable for many discrete values
Fall-through	Not possible	Possible (if break omitted)
Use Case	Range checks, complex logic	Menu selection, day/month mapping
Speed	Linear evaluation	Jump table — faster for many cases

§4 — Loops in C

(A) for Loop

```
for (init; condition; update) { // body }
```

init ↓ condition → FALSE → exit ↓ TRUE
body ↓ update ↓ (back to condition)

```
// Print 1 to 5
for (int i = 1; i ≤ 5; i++) {
    printf("%d ", i);
}
// Output: 1 2 3 4 5
```

When to use:

When the number of iterations is **known** in advance.

(B) while Loop

```
init; while (condition) { // body update;
}
```

condition → FALSE → exit ↓ TRUE body +
update ↓ (back to condition)

```
// Print 1 to 5
int i = 1;
while (i ≤ 5) {
    printf("%d ", i);
    i++;
}
// Output: 1 2 3 4 5
```

When to use:

When the number of iterations is **not known** — check first, then execute.

(C) do-while Loop

```
init; do { // body (runs at least ONCE)
update; } while (condition);
```

★ Key Feature

The loop body executes **at least once**, even if the condition is false from the beginning — because the condition is checked **AFTER** the body.

```
// Print 1 to 5
int i = 1;
do {
    printf("%d ", i);
    i++;
} while (i ≤ 5);
// Output: 1 2 3 4 5

// Even with false condition:
int x = 10;
do {
    printf("runs once");
} while (x < 5); // false, but still ran!
```

for vs while vs do-while — Master Comparison

Feature	for	while	do-while
Condition Check	Before body (entry-controlled)	Before body (entry-controlled)	After body (exit-controlled)
Minimum Executions	0 times	0 times	1 time (always)
Use Case	Known iteration count	Unknown count, check first	Must execute at least once

Init & Update	Built into header	Written separately	Written separately
Syntax Ends With	}	}	} while(cond);
Best For	Counting loops, arrays	File/input reading	Menu-driven programs

⚡ Infinite Loop Warning

A loop becomes infinite if the condition never becomes false. Always ensure the update expression changes the loop variable!

`for(;;) { }` | `while(1) { }` — both are infinite loops (sometimes intentional with `break`).

§5 — Loop Control Statements

(A) break

Definition

Immediately **terminates** the nearest enclosing loop or switch statement and transfers control to the statement after the loop.

```
break;
```

```
for (int i = 1; i ≤ 10; i++) {  
    if (i == 5) break; // stop at 5  
    printf("%d ", i);  
}  
// Output: 1 2 3 4
```

(B) continue

Definition

Skips the **remaining statements** in the current iteration and jumps back to the loop's condition check (or update in for-loop).

```
continue;
```

```
for (int i = 1; i ≤ 5; i++) {  
    if (i == 3) continue; // skip 3  
    printf("%d ", i);  
}  
// Output: 1 2 4 5
```

break vs continue — Comparison

Feature	break	continue
Purpose	Exits the loop entirely	Skips current iteration only
Effect on Loop	Loop terminates immediately	Loop continues with next iteration
Works In	Loops AND switch statements	Loops only
Control Goes To	Statement after the loop	Loop condition (or update in for)
Usage	Exit on error/found condition	Skip specific values

§6 — Problem Solving Using Loops

Sum of N Numbers

```
int sum = 0;  
for(int i=1;i≤n;i++)  
    sum += i;  
printf("%d", sum);
```

Factorial of N

```
int f = 1;  
for(int i=1;i≤n;i++)  
    f *= i;  
printf("%d", f);
```

Prime Check

```
int flag = 1;  
for(int i=2;i<n;i++)  
    if(n%i==0)  
        {flag=0; break;}  
if(flag) printf("Prime");
```

Reverse a Number

```
int rev = 0;  
while(n ≠ 0) {  
    rev = rev*10 + n%10;  
    n /= 10;  
}
```

Multiplication Table

```
for(int i=1;i≤10;i++)  
    printf("%d x %d = %d\n",  
        n, i, n*i);
```

Fibonacci Series

```
int a=0, b=1, c;  
printf("%d %d ",a,b);  
for(int i=2;i<n;i++){  
    c=a+b; printf("%d ",c);  
}
```

```
}  
printf("%d", rev);
```

```
a=b; b=c;  
}
```

§7 — Nested Loops & Patterns

Definition

A loop placed inside the body of another loop. The **inner loop completes all its iterations** for every single iteration of the outer loop.

```
for (i=1; i≤rows; i++) { // outer:  
    controls rows for (j=1; j≤i; j++) { //  
        inner: controls cols printf("* "); }  
    printf("\n"); }
```

Total iterations

= outer iterations × inner iterations

Right Triangle Pattern Output

Row 1: * Row 2: * * Row 3: * * * Row 4: *
* * * Code: for(i=1; i≤4; i++){ for(j=1;
j≤i; j++) printf("* "); printf("\n"); }

Number Pattern

```
1  
1 2  
1 2 3  
for(i=1;i≤3;i++){  
    for(j=1;j≤i;j++)  
        printf("%d ",j);  
    printf("\n");  
}
```

§10 — Formula / Syntax Reference Box

All Conditional Syntaxes

```
/* if */ if(cond){ stmts; } /* if-else */  
if(cond){ stmts; } else { stmts; } /*  
else-if ladder */ if (c1){ ... } else  
if(c2){ ... } else if(c3){ ... } else {  
... } /* switch */ switch(expr){ case v1:  
stmt; break; case v2: stmt; break;  
default: stmt; }
```

All Loop Syntaxes

```
/* for loop */ for(init; cond; update){  
body; } /* while loop */ init; while(cond)  
{ body; update; } /* do-while loop */  
init; do{ body; update; }while(cond); /*  
nested loop */ for(i=1;i≤r;i++){  
for(j=1;j≤c;j++){ body; } }
```

Loop Control Syntax

`break;` → exits loop | `continue;` → skip iteration | `goto label;` → jump to label (avoid in practice)

§9 — Key Terms & Definitions (30 terms)

Control Structure → Directs execution flow of a program

Selection → Chooses a path based on condition

Condition → Expression that evaluates to true or false

else → Block executed when if condition is false

Nested if → if statement inside another if

case → Label for a specific value in switch

break → Terminates current loop or switch immediately

Loop → A repetition construct in programming

while loop → Entry-controlled loop; checks condition first

Loop variable → Variable that controls loop termination

Increment/Decrement → Update step of loop variable

Nested Loop → Loop inside another loop

Inner Loop → Loop inside the outer loop

Exit-Controlled → Condition checked after executing body

Sequential Execution → Statements run one after another

Iteration → Repeats a block of statements

if statement → Executes block only when condition is true

else-if ladder → Chain of if-else for multiple conditions

switch → Multi-branch selection using integer/char value

default → Switch case when no case matches

continue → Skips rest of current iteration

for loop → Loop with init, condition, update in header

do-while loop → Exit-controlled loop; runs body at least once

Initialization → Setting starting value of loop variable

Infinite Loop → Loop whose condition never becomes false

Outer Loop → The enclosing (parent) loop

Entry-Controlled → Condition checked before executing body

Fall-through → Execution continues to next case in switch

Relational Operator → Compares two values (<, >, ==, !=, <=, >=)

Logical Operator → Combines conditions (&&, ||, !)

Algorithm → Step-by-step procedure to solve a problem

Flowchart → Diagrammatic representation of algorithm

Factorial → Product of all integers from 1 to n

Prime Number → Number divisible only by 1 and itself

Modulus (%) → Returns remainder of integer division

§11 — Flashcards (35 Q&A)

Q: What is a control structure?

A: A construct that controls the order of statement execution.

Q: Name the three types of control structures.

A: Sequential, Selection (conditional), Iteration (loop).

Q: What is a loop?

A: A structure that repeats a block of code while a condition is true.

Q: Difference between while and do-while?

A: do-while executes the body at least once; while may not execute at all.

Q: What does break do in a loop?

A: Immediately exits the current loop or switch statement.

Q: What does continue do?

A: Skips the rest of the current iteration and jumps to the next.

Q: What is the default case in switch?

A: The block that executes when no other case matches.

Q: What happens if break is omitted in switch?

A: Execution falls through to the next case (fall-through).

Q: What type of values can switch use?

A: Integer or character constants only (not float or string).

Q: Which loop is exit-controlled?

A: do-while loop — condition is checked after body executes.

Q: What is an entry-controlled loop?

A: A loop where the condition is checked before executing the body (for, while).

Q: How many times does do-while run with a false condition?

A: Exactly once — body always executes before first condition check.

Q: What are the three parts of a for loop header?

A: Initialization; condition; update/increment.

Q: What is an infinite loop?

A: A loop whose termination condition never becomes false.

Q: What is a nested loop?

A: A loop that is placed inside the body of another loop.

Q: In a nested loop, how many total iterations occur?

A: Outer iterations × inner iterations.

Q: When should you use a for loop vs while loop?

A: for when count is known; while when count is unknown.

Q: Can break be used in a switch statement?

A: Yes — break exits the switch block.

Q: What is the difference between = and == in C?

A: = assigns a value; == checks equality.

Q: What is sequential execution?

A: Statements execute one by one from top to bottom in order written.

Q: What does else-if ladder do?

A: Tests multiple conditions in sequence; first true block executes.

Q: How do you check if a number is even?

A: Use `n % 2 == 0`.

Q: What is the modulus operator?

A: % — returns the remainder of integer division.

Q: Which control structure is best for menu-driven programs?

A: do-while (runs at least once) combined with switch.

Q: What is fall-through in switch?

Q: Can you use float in a switch expression?

A: Execution continues to the next case when break is missing.

A: No — switch only works with integer or character types.

Q: What does continue do in a for loop specifically?

A: Skips to the update expression, then re-checks condition.

Q: What is the output of: for(i=0;i<5;i++) if(i==3) break; printf("%d",i);?

A: 3 — loop exits when i equals 3.

Q: What is the purpose of the update expression in a for loop?

A: It modifies the loop variable to eventually terminate the loop.

Q: What semicolons are needed in a for loop header?

A: Two semicolons: for(init ; condition ; update).

Q: Nested if vs else-if ladder — when to use which?

A: Nested if for multiple simultaneous conditions; ladder for mutually exclusive conditions.

Q: How to print a right-angled star triangle with 4 rows?

A: Outer loop 1 to 4; inner loop 1 to i; print * in inner, newline after inner.

Q: What is the logical AND operator in C?

A: && — true only if BOTH conditions are true.

Q: What happens when condition is omitted in for(;;)?

A: Condition is treated as always true — infinite loop.

Q: Can continue be used in a switch statement?

A: No — continue applies to loops only, not switch.

§12 — Exam Tips

✗ Common Mistakes to AVOID

- Using `=` instead of `==` in conditions.
- Missing semicolon after `do-while : { }`
`while(cond);`
- Forgetting `break` in switch causing fall-through.
- Off-by-one errors in loop bounds (`<` vs `≤`).
- Infinite loop — loop variable never updated.
- Using float/string in switch expression.
- Confusing `break` (exits loop) with `continue` (next iteration).
- Declaring variables inside loops without initialization.
- Nested if without proper braces — logic errors.
- Not printing newline `\n` in pattern problems.

✓ Writing Tips for Exams

- Always write correct syntax with proper braces `{ }` .
- Use proper indentation to show structure.
- Add comments for clarity in exam answers.
- Draw simple flowcharts for if-else and loops.
- State the output after your code.
- For switch, always include `break` and `default` .
- For loop problems, trace manually: write iteration table.
- Highlight keywords: `if`, `else`, `for`, `while`, `break` .
- Use meaningful variable names: `i`, `j`, `sum`, `count`, `n` .
- Verify: does your loop handle `n=0` or `n=1` correctly?

§8 — Master Comparison Tables

Sequential vs Conditional vs Loop

Aspect	Sequential	Conditional (Selection)	Loop (Iteration)
Execution	All statements once, in order	Only matched block executes	Block repeats multiple times
Decision Making	None	Yes — based on condition	Yes — loop condition
Repetition	None	None	Yes
Keywords	—	if, else, switch	for, while, do-while
Example Use	Simple calculations	Grade classification	Sum of series, patterns

if-else vs switch (Detailed)

Criterion	if-else	switch
Data Types	int, float, char, expressions	int, char only

Conditions	Range: $x > 10$, $x < 0$, etc.	Exact equality only
Fall-through	Not applicable	Yes, if break omitted
Nested Support	Yes	Yes (limited)
Speed	Slower for many conditions	Faster (jump table)

for vs while vs do-while (Summary)

Property	for	while	do-while
Type	Entry-controlled	Entry-controlled	Exit-controlled
Min executions	0	0	1
Semicolons in header	2	1	1 (at end)
Ideal for	Known count	Unknown count	Must run once

break vs continue

Property	break	continue
Action	Exits entire loop	Skips current iteration
Next step	Statement after loop	Next iteration / condition check
Works in	Loops + switch	Loops only



QUICK REVISION SHEET

No explanations · Only essentials · Syntax + Rules + Differences

🔑 Keywords

- `if` , `else` , `switch` , `case` , `default`
- `for` , `while` , `do`
- `break` , `continue`

12 Operators in Conditions

- Relational: `<` `>` `≤` `≥` `=` `≠`
- Logical: `&&` (AND) `||` (OR) `!` (NOT)
- Assignment vs Equality: `=` assigns, `==` checks

📌 if Rules

- Condition in parentheses is mandatory
- Braces optional for single statement
- `else` always pairs with nearest `if`

📌 switch Rules

- Expression must be `int` or `char`
- `break` after each case (else fall-through)
- default is optional but recommended
- Duplicate case values NOT allowed

📌 for Loop Rules

- Syntax: `for(init; cond; update)`
- Two semicolons in header — always
- Any part can be omitted (but semicolons stay)
- Entry-controlled; can run 0 times

📌 while Loop Rules

- Condition checked BEFORE body
- init and update written separately
- Runs 0 times if condition is initially false
- Update inside body is mandatory

📌 do-while Rules

- Body executes BEFORE condition check
- Runs at least once always
- Semicolon after `while(cond);` is mandatory
- Exit-controlled loop

📌 break & continue Rules

- `break` — exits loop/switch entirely
- `continue` — skips to next iteration
- `continue` does NOT work in switch
- In nested loops, `break/continue` affect only innermost loop



Key Differences at a Glance

for vs while

`for` → count known
`while` → count unknown
 Both → entry-controlled

while vs do-while

`while` → 0 executions possible
`do-while` → always ≥1 execution
`do-while` → exit-controlled

if vs switch

`if` → any condition/range
`switch` → equality, `int/char` only
`switch` → faster for many values

break vs continue

`break` → exits loop
`continue` → next iteration
`break` works in switch too

Sequential vs Loop

sequential → no repetition
 loop → repeats block
 loop has condition check

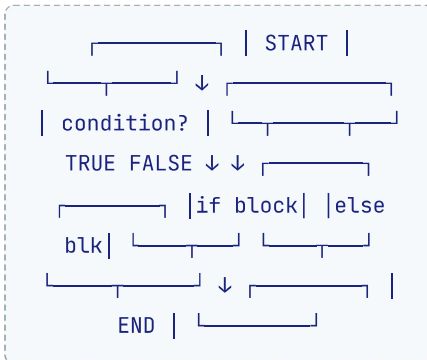
Nested Loop Count

outer × inner = total
 inner restarts each outer
`break` affects only innermost

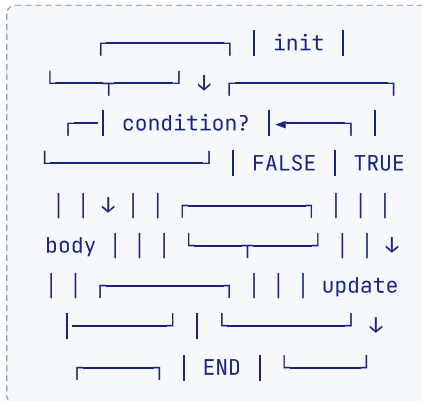
✅ **Must-Remember Formulas for Common Problems:** Sum 1 to $n = n(n+1)/2$ | Factorial $n! = 1 \times 2 \times \dots \times n$ | Reverse: $rem = n \% 10$; $rev = rev * 10 + rem$; $n /= 10$ | Prime: check divisors 2 to $n-1$, $flag = 0$ if divisible

Visual Flow Reference — All Constructs

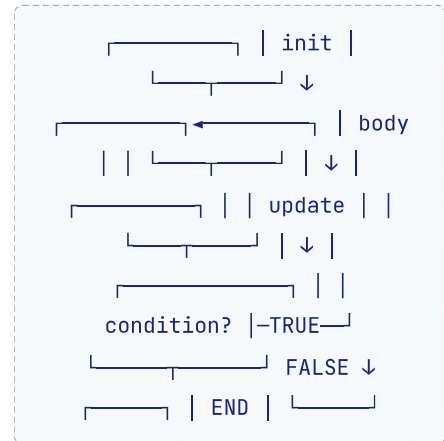
if-else Flow



for / while Loop Flow



do-while Loop Flow



switch Statement Flow

expression | — case 1 match? → YES → execute case 1 → break → EXIT switch | — case 2 match? → YES → execute case 2 → break → EXIT switch | — case 3 match? → YES → execute case 3 → break → EXIT switch | — none match? —————→ execute default → EXIT switch
 Δ If break is MISSING: case 1 executes → falls into case 2 → falls into case 3... (fall-through!)

Pattern Reference — Quick Guide

Right Triangle (*)

```

*
* *
* * *
* * * *
  
```

for j=1 to i

Number Triangle

```

1
1 2
1 2 3
1 2 3 4
  
```

print j (j=1 to i)

Square Pattern

```

* * * *
* * * *
* * * *
* * * *
  
```

j=1 to n (fixed)

Row Numbers

```

1
2 2
3 3 3
4 4 4 4
  
```

print i (j=1 to i)

Iteration Trace Table Example

Trace: `for(i=1; i≤4; i++) sum += i;` (starting sum=0)

Iteration	i	Condition (i≤4)	sum += i	sum after
1	1	TRUE	0+1	1
2	2	TRUE	1+2	3
3	3	TRUE	3+3	6
4	4	TRUE	6+4	10
5	5	FALSE	—	Loop exits, sum = 10

